

GANGA INSTITUTE OF TECHNOLOGY & MANAGEMENT JHAJJAR, HARYANA

VLSI (Very Large Scale Integration) Internship– Task 1

MAINCRAFTS TECHNOLOGY INTERNSHIP:

**“Introduction to VLSI Design Flow and
Basic Digital Logic Implementation”**

**SUBMITTED TO:
MAINCRAFTS TECHNOLOGY**

**SUBMITTED BY :
NAME - ANKIT KUMAR
COLLEGE ROLL – 24ECE006
PROGRAM - VLSI (Very Large Scale
Integration) Internship
TASK - 1**

Objective

The objective of this internship (task-1) is to understand the VLSI design flow, study basic digital logic gates, verify their truth tables through simulation, and implement a Half Adder circuit using logic gates.

What is VLSI?

VLSI (Very Large Scale Integration) is the process of designing and manufacturing Integrated Circuits (ICs) by integrating millions or even billions of transistors onto a single silicon chip. It enables the development of compact, high-speed, and low-power electronic devices.

Use of VLSI

- Computers and laptops
- Smartphones and tablets
- Consumer electronics
- Automotive electronics
- Communication systems
- Medical equipment
- Aerospace and defence
- Industrial automation
- Internet of things
- Artificial intelligence and data centers

Advantages of VLSI

- Compact size
- High speed
- Low power consumption
- High reliability
- Lower production cost
- Improved performance
- Greater functionality

VLSI Chip Design : From Idea to Fabrication

Designing a VLSI chip is a long and systematic process that transforms an idea into a physical silicon chip. This process is known as the VLSI Design Flow.

i. System Specification (VLSI Design Flow)

System Specification is the first stage of the VLSI design flow. In this stage, the customer or designer defines what the chip should do and lists all the requirements before starting the actual design.

ii. Architecture Design

Architecture Design is the second stage of the VLSI design flow. In this stage, engineers decide how the chip will perform the required functions defined during the System Specification stage.

iii. RTL Design (Register Transfer Level Design)

RTL (Register Transfer Level) Design is the third stage of the VLSI design flow. In this stage, the architecture of the chip is converted into a hardware description using a Hardware Description Language (HDL), such as Verilog or VHDL

iv. Functional Verification (VLSI Design Flow)

Functional Verification is the fourth stage of the VLSI design flow. In this stage, engineers verify that the RTL (Register Transfer Level) design works correctly according to the system specifications before moving to hardware implementation.

v. Synthesis (VLSI Design Flow)

Synthesis is the fifth stage of the VLSI design flow. In this stage, the RTL (Register Transfer Level) code written in Verilog or VHDL is converted into a gate-level netlist using standard logic gates such as AND, OR, NOT, NAND, NOR, XOR, flip-flops, and multiplexers.

vi. Physical Design (VLSI Design Flow)

Physical Design (PD) is the sixth stage of the VLSI design flow. In this stage, the gate-level netlist generated during synthesis is converted into the physical layout of the integrated circuit (IC). This layout specifies the exact placement of transistors, logic gates, and the metal wires that connect them.

vii. Timing and Power Analysis (VLSI Design Flow)

Timing and Power Analysis is an important stage in the VLSI design flow, usually performed after Physical Design and before Fabrication. In this stage, engineers verify that the chip operates at the required speed and consumes acceptable power.

viii. Fabrication (VLSI Design Flow)

Fabrication is the final manufacturing stage of the VLSI design flow. In this stage, the physical layout (GDSII file) created during Physical Design is used to manufacture the integrated circuit (IC) on a silicon wafer using semiconductor manufacturing processes.

ix. Testing and Validation (VLSI Design Flow)

Testing and Validation is the final stage of the VLSI design flow. After the chip is fabricated, it is tested to ensure that it functions correctly, meets all design specifications, and is free from manufacturing defects.

LOGIC GATES

i. AND Gate

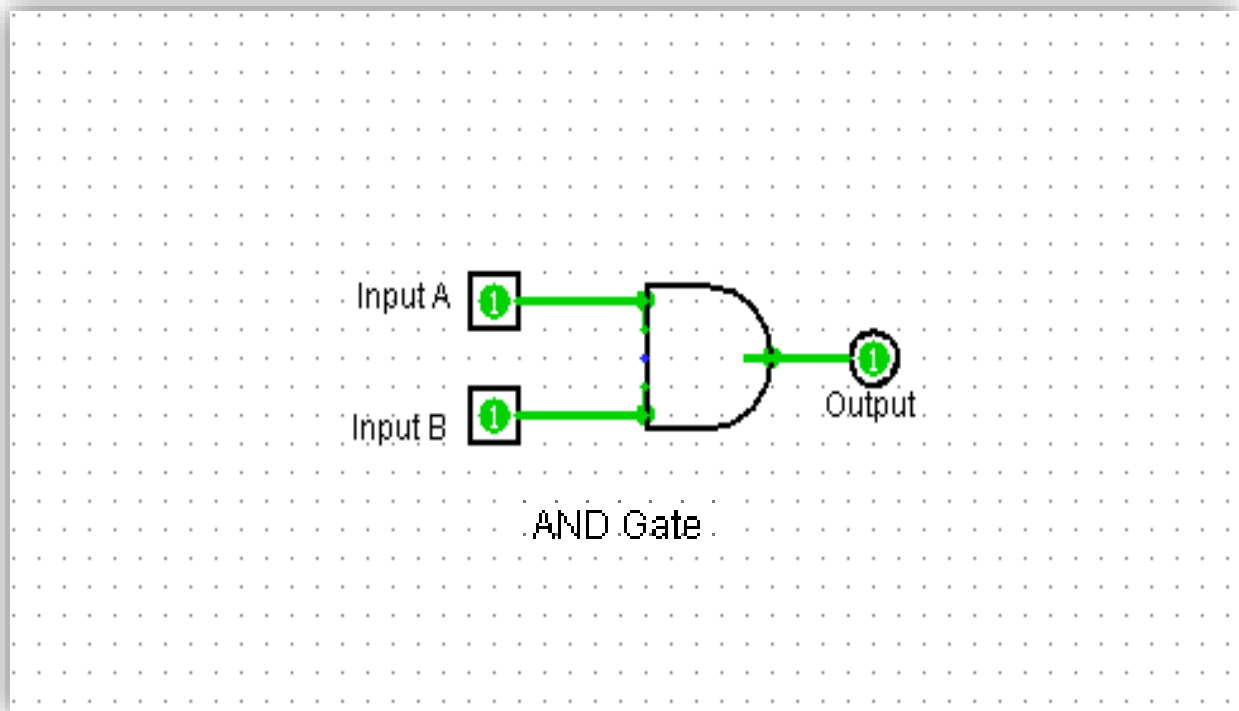
An AND gate is a basic digital logic gate that produces an output of 1 (HIGH) only when all its inputs are 1 (HIGH). If any input is 0 (LOW), the output is 0.

Truth Table

Input A	Input B	Output Y = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Expression : $Y = A \cdot B$

Simulation:



ii. OR Gate

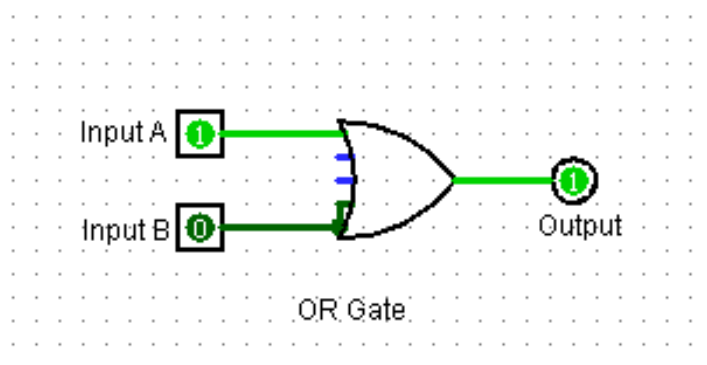
An OR gate is a basic digital logic gate that produces an output of 1 (HIGH) when at least one of its inputs is 1 (HIGH). The output is 0 (LOW) only when all inputs are 0.

Truth Table :

Input A	Input B	Output Y = A AND B
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Expression : $Y=A+B$

Simulation:



iii. NOT Gate

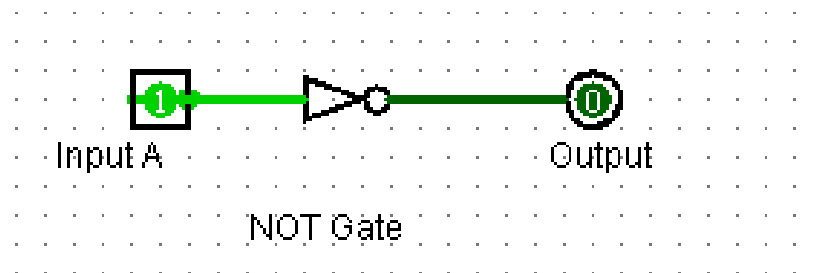
A NOT gate is a basic digital logic gate that performs the operation of inversion (complement). It has only one input and one output. The output is always the opposite of the input. It is also called an Inverter.

Truth Table :

Input A	Output Y = $\bar{A}$
0	1
1	0

Boolean Expression : $Y = A'$

Simulation :



iv. NAND Gate

A NAND gate is a combination of an AND gate followed by a NOT gate. The word NAND means NOT + AND.

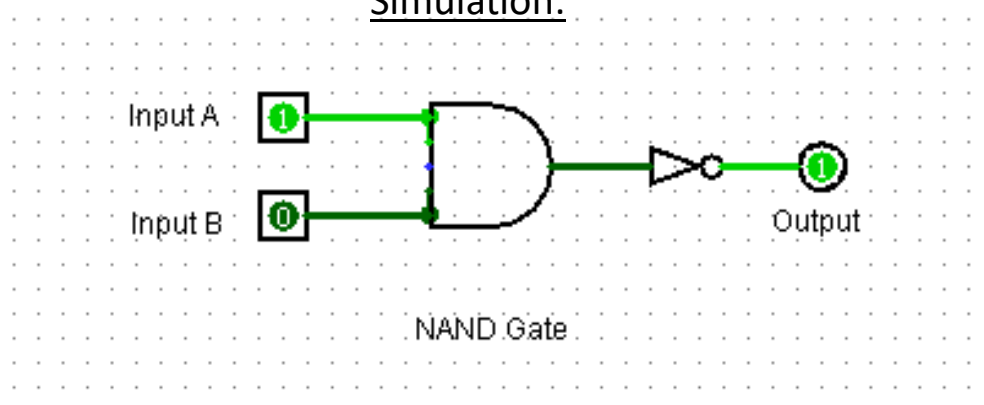
It gives an output of 0 (LOW) only when all inputs are 1 (HIGH). For all other input combinations, the output is 1 (HIGH).

Truth Table :

Input A	Input B	AND Output	NAND Output $Y = (A \cdot B)'$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Boolean Expression : $Y = (A \cdot B)'$

Simulation:



v. NOR Gate

A NOR gate is a combination of an OR gate followed by a NOT gate. The word NOR means NOT + OR.

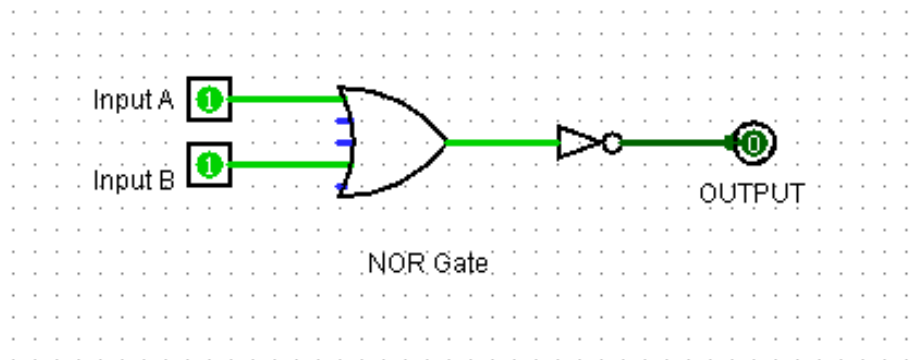
A NOR gate gives an output of 1 (HIGH) only when all inputs are 0 (LOW). If any input is 1 (HIGH), the output becomes 0 (LOW).

Truth Table :

Input A	Input B	AND Output	NAND Output $Y = (A \cdot B)'$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Boolean Expression : $Y = (A+B)'$

Simulation :



vi. XOR Gate (Exclusive OR Gate)

An XOR gate (Exclusive OR gate) is a digital logic gate that produces an output of 1 (HIGH) when the inputs are different. If both inputs are the same, the output is 0 (LOW).

The word Exclusive OR means that the output is HIGH when only one input is HIGH.

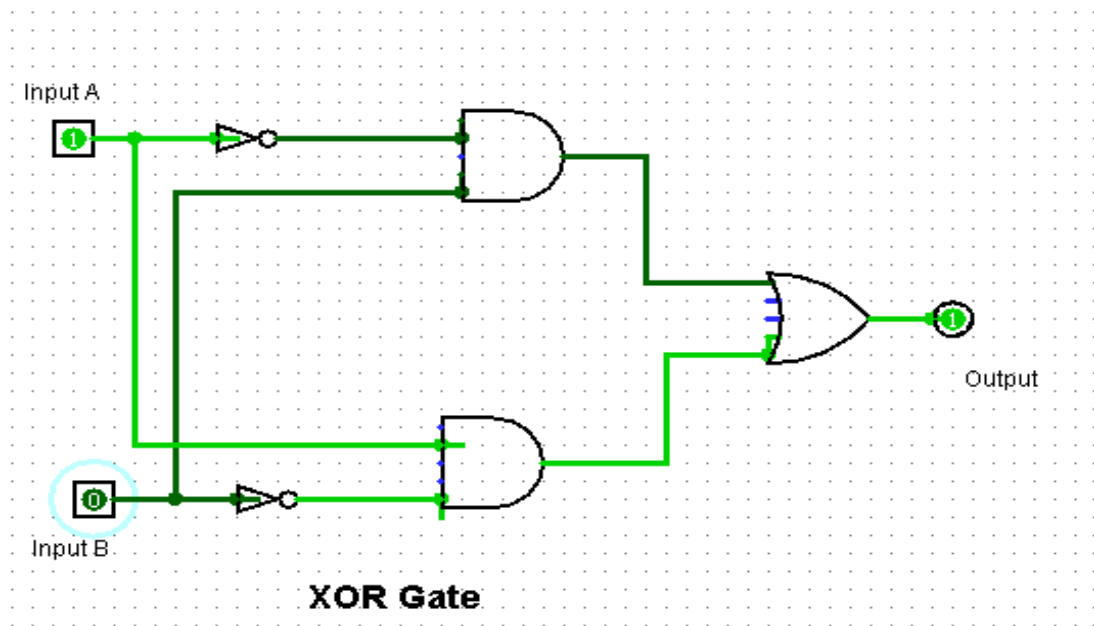
Truth Table :

Input A	Input B	Output $Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Boolean Expression: $Y = A \oplus B$

Expanded form: $Y = \bar{A}B + A\bar{B}$

Simulation :



vii. XNOR Gate (Exclusive NOR Gate) :

An XNOR gate (Exclusive NOR gate) is a digital logic gate that produces an output of 1 (HIGH) when all inputs are the same. It gives an output of 0 (LOW) when the inputs are different.

XNOR is the opposite of XOR gate. It is also called an Equality Gate because it checks whether the inputs are equal or not.

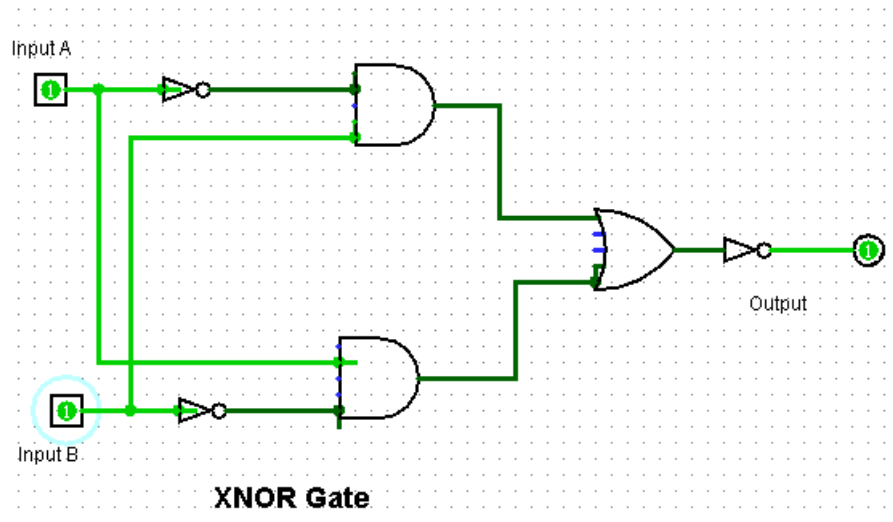
Truth Table :

Input A	Input B	Output $Y = A \oplus B$
0	0	1
0	1	0
1	0	0
1	1	1

Boolean Expression: $(A \oplus B)'$

Expanded form: $Y = AB + \bar{A}\bar{B}$

Simulation :



Half Adder

A Half Adder is a combinational digital circuit used to perform the addition of two binary bits. It has two inputs and two outputs.

The two inputs are:

- A → First binary input
- B → Second binary input

The two outputs are:

- Sum (S) → Gives the addition result
- Carry (C) → Gives the carry generated during addition

Boolean Expression :

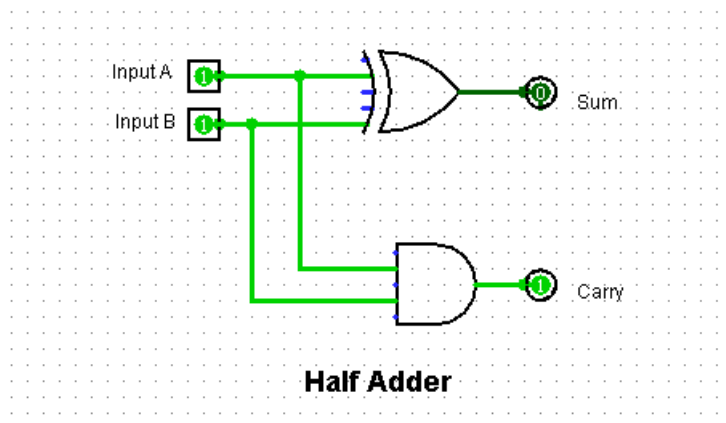
Sum = $A \oplus B$ (Sum is generated by XOR gate)

Carry = $A \cdot B$ (Carry is generated by AND gate)

Truth Table :

Input A	Input B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Simulation :



Full Adder

A Full Adder is a combinational digital circuit that is used to add three binary bits. It is an extension of a Half Adder because it can add a previous carry input along with two binary inputs.

A Full Adder has:

Inputs:

1. A → First binary input
2. B → Second binary input
3. Cin (Carry Input) → Previous carry from the lower bit addition

Outputs:

1. Sum (S) → Result of binary addition
2. Cout (Carry Output) → Carry generated after addition

Boolean Expression:

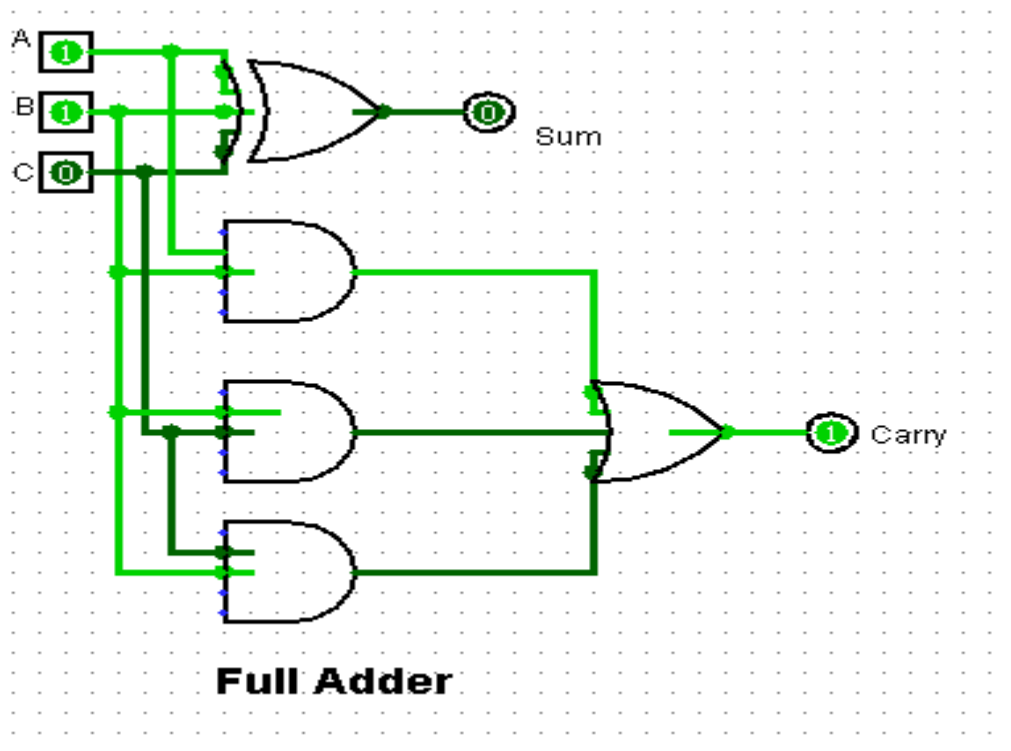
$$\text{Sum: } A \oplus B \oplus \text{Cin}$$

$$\text{Carry Output : } AB + \text{BCin} + \text{ACin}$$

Truth Table

Input A	Input B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Simulation :



Tools Used : Logisim, MS Word